Project 4:- Maven Selenium Project – MyMavenSeleniumApp02

Objective

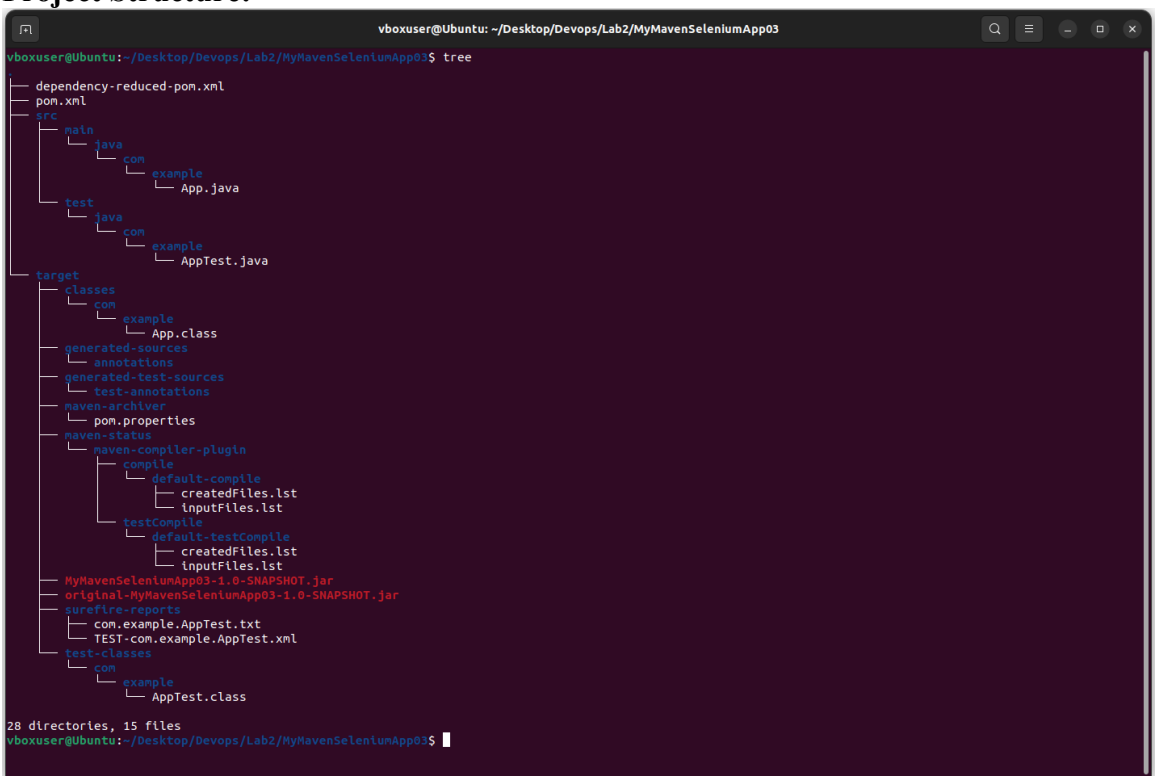
The goal of this project is to use Maven and Selenium WebDriver to write a Java program that opens a browser, finds specific elements on a webpage, and performs a series of automated interactions without any human input.

Step 1: Create a Maven Project Using Terminal

Open a terminal window and enter the following command to scaffold the Maven project:

```
mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenSeleniumApp02
  -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

Project Structure:



Step 2: Add Selenium Dependency in pom.xml

In the pom.xml file, locate the <dependencies> section and insert the entry below:

```
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>4.29.0</version>
</dependency>
```

Step 3: Configure Maven Build Plugins

Include the Maven compiler plugin in the <build> section to specify which Java version to use:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
```

```
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.8.1</version>
            <configuration>
              <source>1.8</source>
              <target>1.8</target>
            </configuration>
          </plugin>
        </plugins>
      </build>
```

Step 4: Inspect Elements in the Website

Launch the browser and visit the target site. Right-click on any element you plan to automate and choose Inspect from the context menu. Record the relevant attributes like id, class, or name — these will be referenced in the Java code.

Step 5: Create App.java

Go to the path below and write the automation logic in App.java:

```
src/main/java/com/example/App.java
```

```java
package com.example;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class App {
    public static void main(String[] args) throws InterruptedException {

        WebDriver driver = new ChromeDriver();
        Thread.sleep(3000);

        driver.get("http://practicetestautomation.com/practice-test-login/");
        Thread.sleep(3000);

        driver.manage().window().maximize();
        Thread.sleep(3000);

        driver.findElement(By.id("username")).sendKeys("student");
        Thread.sleep(3000);

        driver.findElement(By.id("password")).sendKeys("Password123");
        Thread.sleep(3000);

        driver.findElement(By.id("submit")).click();
        Thread.sleep(3000);

        driver.quit();
    }
}
```

Step 6: Build the Maven Project

Use the following command to clean any previous build and recompile the project:

```
mvn clean compile
```

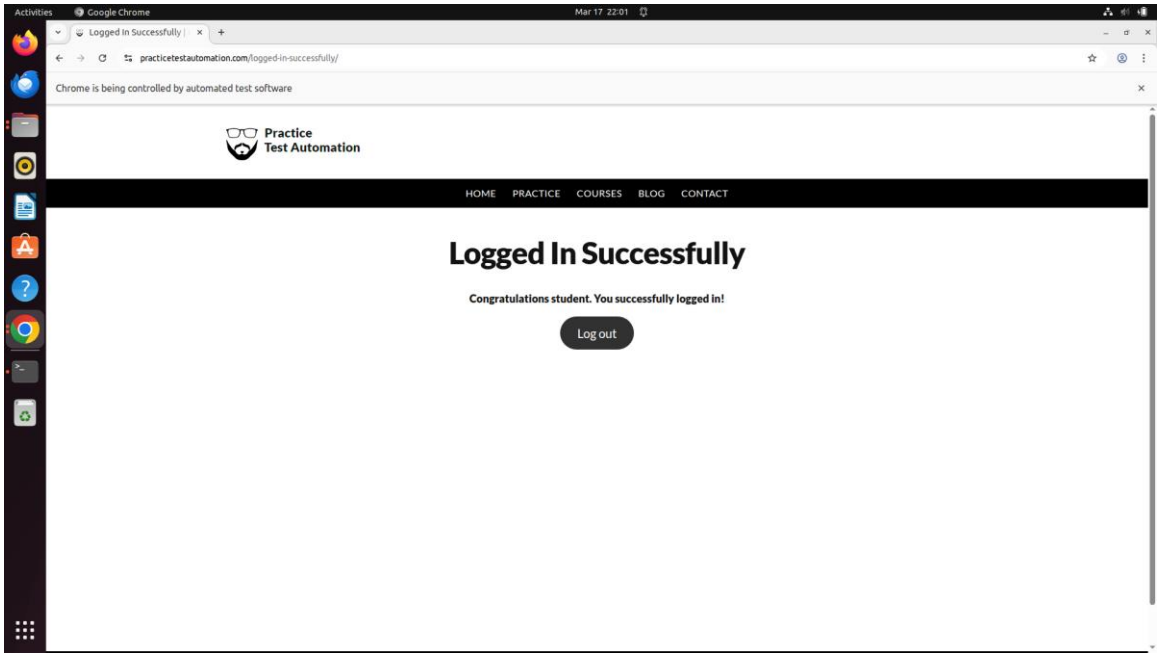
Step 7: Run the Application

Start the application by running:

```
mvn exec:java
```

Output

The program runs as expected, completing all the browser interactions defined in the script.



Conclusion

Through this project, we successfully set up a Maven-Selenium environment and automated a series of browser actions end-to-end. The entire workflow ran without requiring any manual steps.